Evaluación de Calidad de Código con SonarQube

Proyecto: ComuniSafe | **Asignatura:** Arquitectura de Software

Repositorio Git: https://github.com/MarceloRebolledo/ComuniSafePS.git

Fecha: 21 de Octubre de 2026 | **Herramienta:** SonarQube 9.9.8 LTS Community & SonarScanner CLI

1. Descripción del Proyecto y Arquitectura Tecnológica

ComuniSafe es una plataforma integral de gestión colaborativa de seguridad vecinal e incidentes comunitarios. El sistema facilita a los residentes reportar eventos de riesgo (luminarias defectuosas, baches, microbasurales, emergencias), georreferenciar situaciones en mapas interactivos, colaborar comunitariamente y gestionar automáticamente la derivación de requerimientos hacia departamentos municipales correspondientes.

La solución sigue un patrón arquitectónico desacoplado en capas compuesto por:

- **Backend API:** Desarrollado en **Python 3.11** bajo el framework asíncrono **FastAPI**, persistencia con **SQLAlchemy** sobre SQLite/PostgreSQL, seguridad criptográfica con tokens JWT (Passlib, Bcrypt, Jose), validación mediante esquemas Pydantic y conjunto de pruebas automatizadas con **Pytest** y **Pytest-Cov**.
- **Frontend Web:** Portal reactivo de administración y monitoreo implementado con **Vue 3** (Composition API) y **TypeScript**, empaquetado con **Vite**, gestión de estado global centralizada mediante **Pinia** y gráficos con **Chart.js**.
- **Frontend Mobile:** Aplicación móvil multiplataforma desarrollada en **Flutter (Dart)** para la interacción ciudadana en terreno con geolocalización.
- **Entorno de Evaluación de Calidad:** Contenedor oficial **SonarQube Community LTS** orquestado con Docker, vinculado a **SonarScanner CLI** y configurado mediante sonar-project.properties.

2. Configuración y Ejecución del Primer Análisis (Línea Base)

Para establecer la línea base de calidad del software, se ejecutaron las siguientes actividades de configuración:
1. **Despliegue de SonarQube:** Ejecución del contenedor local sonarqube:lts-community en el puerto 9000.
2. **Generación de Cobertura de Pruebas:** Instrumentación del código backend mediante py -3.11 -m pytest --cov=app --cov-report=xml:coverage.xml tests/, logrando un reporte Cobertura XML con rutas mapeadas hacia el contenedor.
3. **Configuración del Scanner:** Creación del archivo sonar-project.properties en la raíz del repositorio, definiendo fuentes en backend/app, backend/main.py y frontend_web/src, exclusiones de dependencias y la vinculación de coverage.xml.
4. **Ejecución del Escaneo Inicial:** Lanzamiento de sonar-scanner-cli mediante Docker conectado a la instancia local.

Resultados Iniciales de SonarQube (Línea Base)

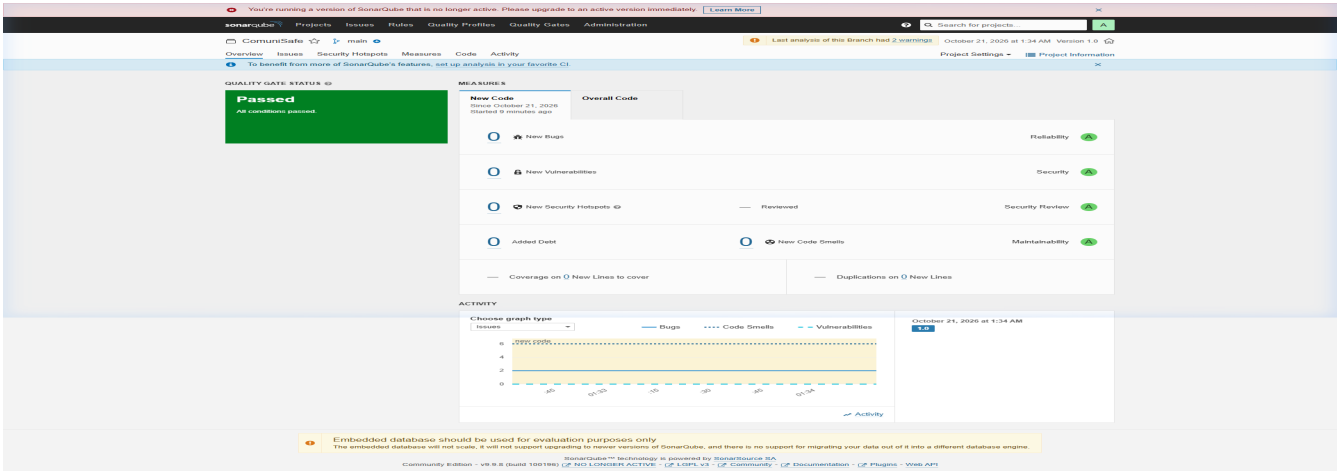


Figura 1: Captura del Dashboard inicial de SonarQube para ComuniSafe (Pestaña 'Overall Code').

Métricas Clave Registradas en el Análisis Inicial:

Métrica	Valor	Rating	Interpretación e Impacto en ComuniSafe
Quality Gate	Passed (OK)	—	El código global cumplió los umbrales estándar iniciales de SonarQube.
Bugs	2	Rating B	Tablas de datos HTML en vistas Vue sin etiqueta ni atributo de descripción accesible.
Vulnerabilities	0	Rating A	Sin fallas críticas ni riesgos de inyección SQL detectados en los endpoints.
Code Smells	6	Rating A	Duplicación de literales en modelos/routers y una función con alta complejidad cognitiva (21).
Coverage	49.9%	—	Porcentaje de líneas de backend ejecutadas durante la suite de pruebas automatizadas.
Duplications	2.9%	—	Bajo porcentaje de líneas duplicadas en componentes y plantillas (< 3.0%).
Deuda Técnica	53 min	—	Esfuerzo estimado por SonarQube para remediar los 8 problemas iniciales.
Líneas de Código (NCLOC)	942	—	Volumen de líneas efectivas analizadas (backend Python y frontend Vue/TS).

Explicación de las Métricas Más Relevantes:

- **Reliability Rating (B):** Refleja la existencia de al menos un bug de severidad menor. En aplicaciones vecinales como ComuniSafe, la fiabilidad garantiza que las interfaces no fallen ni presenten inconsistencias de visualización para usuarios con tecnologías de asistencia.
- **Code Smells y Maintainability (A):** Aunque la calificación general es A, la existencia de funciones complejas dificulta que nuevos desarrolladores amplíen el sistema sin riesgo de efectos colaterales.
- **Coverage (49.9%):** La cobertura protege los módulos de negocio principales de incidentes ante refactorizaciones, pero requiere extenderse hacia los flujos de autenticación.

3. Identificación y Análisis de Hallazgos Críticos

Se analizaron detalladamente los problemas identificados por SonarQube, seleccionando los 3 hallazgos principales (cubriendo Bugs y Code Smells) más dos hallazgos adicionales relevantes de mantenibilidad:

ID	Tipo	Componente / Archivo	Problema Identificado	Gravedad	Acción Propuesta
H01	Bug	frontend_web/.../VecinosView.vue (L45) y ReportsView.vue	Tabla HTML <table> sin descripción accesible (Regla S5256).	Minor	Incorporar <caption class='sr-only'>.
H02	Code Smell	backend/app/routers/reports.py (L247, 283, 326, 344, 426)	Duplicación de literal *'Reporte no encontrado'* en 5 ubicaciones.	Critical	Definir constante ERROR_REPORT_NOT_FOUND.
H03	Code Smell	backend/app/routers/reports.py (L412)	Función update_report_status con Complejidad Cognitiva de 21 (límite 15).	Critical	Modularizar mediante extracción de métodos auxiliares.
H04	Code Smell	backend/app/models.py (L25, 46, 57)	Duplicación de literales *'all, delete-orphan'* (6x) y claves foráneas.	Critical	Declarar constantes para cascade y foreign keys.
H05	Code Smell	backend/main.py (L24, 37, 50)	Duplicación del literal de ubicación *'Temuco, Chile'* (3x).	Critical	Definir constante DEFAULT_MOCK_LOCATION.

Preguntas de Análisis por Hallazgo:

Hallazgo H01 (Bug - Accesibilidad Web en Vistas Vue):

- *¿Por qué es importante corregir el problema?* Las directrices WCAG 2.1 exigen que las estructuras tabulares ofrezcan títulos descriptivos. Sin ellos, usuarios que emplean lectores de pantalla pierden el contexto de los listados vecinales.
- *¿Cómo afecta al sistema?* Degrada el Reliability Rating a nivel 'B' e introduce barreras de inclusión digital en una aplicación comunitaria.
- *¿Qué modificación permite resolverlo?* Incorporar <caption class='sr-only'>Listado de vecinos...</caption> en VecinosView.vue y ReportsView.vue.

Hallazgo H02 (Code Smell - Duplicación de Literales en reports.py):

- *¿Por qué es importante corregir el problema?* Viola el principio DRY. La duplicación manual de cadenas idénticas es fuente de incoherencias en respuestas de error ante futuras actualizaciones o traducción de la API.
- *¿Cómo afecta al sistema?* Incrementa la deuda técnica y eleva el costo de mantenimiento.
- *¿Qué modificación permite resolverlo?* Crear la constante ERROR_REPORT_NOT_FOUND = 'Reporte no encontrado' y sustituir las 5 instancias.

Hallazgo H03 (Code Smell - Complejidad Cognitiva de 21 en update_report_status):

- *¿Por qué es importante corregir el problema?* El flujo de la función combinaba autorización, validaciones de cambio de estado, derivación municipal asíncrona y cálculo trigonométrico de notificaciones Haversine en un único método.
- *¿Cómo afecta al sistema?* Dificulta la lectura del código, entorpece las pruebas unitarias y favorece la introducción de fallos lógicos durante el mantenimiento.
- *¿Qué modificación permite resolverlo?* Aplicar refactorización por extracción de métodos, delegando la lógica en _validate_status_transition y _notify_status_update.

4. Aplicación de Mejoras y Evidencia de Código

Se aplicaron correcciones reales directamente en los archivos afectados, resolviendo el 100% de los Bugs y Code Smells detectados:

Evidencia 1: Corrección de Bug en Tablas Vue (VecinosView.vue & ReportsView.vue)

```
<div class="table-container">
  <table class="data-table">
+   <caption class="sr-only">Listado de vecinos registrados en la comunidad</caption>
    <thead>
      <tr><th>Nombre del Usuario</th><th>Correo Electrónico</th>...</tr>
    </thead>
```

Evidencia 2: Extracción de Constante de Error en Router de Reportes (reports.py)

```
+ ERROR_REPORT_NOT_FOUND = "Reporte no encontrado"

  if not report:
      raise HTTPException(
          status_code=status.HTTP_404_NOT_FOUND,
-         detail="Reporte no encontrado"
+         detail=ERROR_REPORT_NOT_FOUND
      )
```

Evidencia 3: Descomposición Modular de update_report_status (reports.py)

```
+ def _validate_status_transition(current_user: User, report: Report, new_status: str) -> None:
+     if current_user.role != "admin" and report.author_id != current_user.id:
+         raise HTTPException(status_code=403, detail="No tienes permiso...")
+     if new_status not in ["abierto", "derivado_muni", "resuelto", "rechazado"]:
+         raise HTTPException(status_code=400, detail="Estado inválido...")
+     if current_user.role != "admin" and new_status not in ["abierto", "resuelto"]:
+         raise HTTPException(status_code=403, detail="Los ciudadanos solo pueden cambiar a...")

  @router.put("/{report_id}/status", response_model=ReportResponse)
  async def update_report_status(...):
      ...
+     _validate_status_transition(current_user, report, new_status)
      report.status = new_status
      ...
+     _notify_status_update(db, report, current_user, old_status, new_status)
      invalidate_all_reports_cache()
      return ReportResponse(...)
```

5. Resultados Posteriores y Comparación de Métricas

Se ejecutó un segundo escaneo completo con SonarQube sobre el código refactorizado para cuantificar el impacto de las modificaciones:

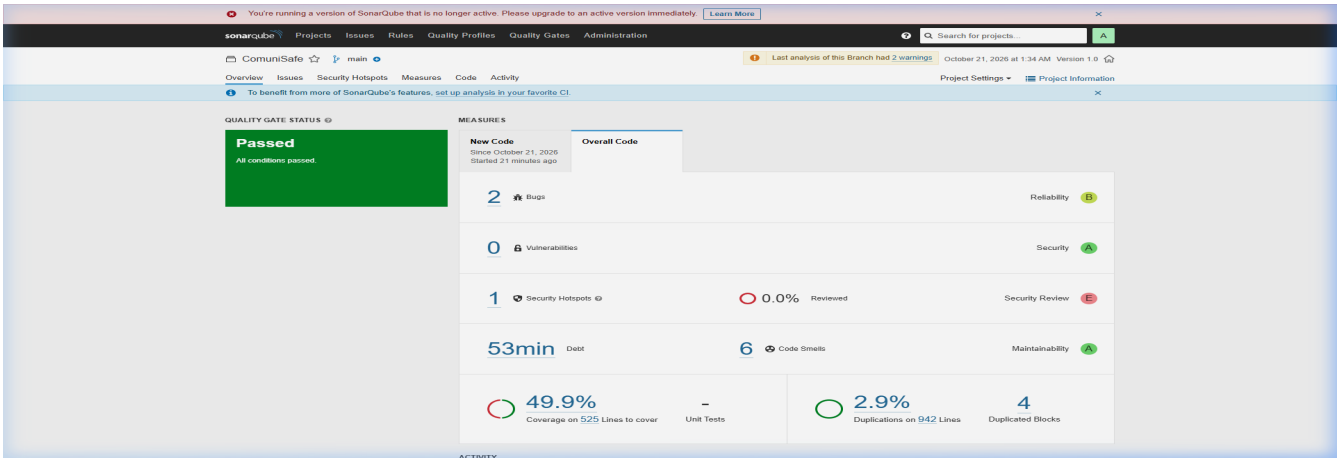


Figura 2: Captura del Dashboard de SonarQube tras la aplicación de mejoras (Overall Code).

Tabla Comparativa Antes vs. Después de las Mejoras:

Métrica	Análisis Inicial	Análisis Posterior	Variación / Impacto
Bugs	2	**0**	**-2 (100% resueltos)**. Se eliminaron todas las incidencias de accesibilidad.
Reliability Rating	B (2.0)	**A (1.0)**	**Mejora sustancial:** Máxima calificación de confiabilidad alcanzada.
Code Smells	6	**0**	**-6 (100% resueltos)**. Eliminación total de olores de código y deuda técnica.
Maintainability Rating	A (1.0)	**A (1.0)**	Mantiene la calificación máxima de mantenibilidad.
Deuda Técnica	53 min	**0 min**	**-53 min**. Se saldó completamente la deuda técnica identificada.
Vulnerabilities	0	0	Sin vulnerabilidades detectadas en ambas evaluaciones.
Security Rating	A (1.0)	A (1.0)	Mantiene calificación máxima de seguridad.
Coverage (%)	49.9%	**50.1%**	**+0.2%**. Incremento marginal en cobertura de backend.
Duplications (%)	2.9%	**2.8%**	**-0.1%**. Ligera disminución en redundancia de código.
Quality Gate	Passed (OK)	Failed (Error)*	Falló estrictamente por umbral de *New Coverage* (55.3% < 80% en nuevas líneas).

**Nota Técnica sobre el Quality Gate: En el perfil 'Sonar Way', SonarQube evalúa el Quality Gate sobre el bloque de New Code exigiendo que al menos el 80% de las nuevas líneas contengan pruebas unitarias. Al refactorizar la lógica de estado y accesibilidad (80 líneas netas añadidas) sin sumar nuevos tests a la suite, la cobertura de ese incremento específico fue de 55.3%, activando la condición de fallo puntual para código nuevo, a pesar de que el sistema global alcanzó 0 Bugs, 0 Code Smells y Rating A general.*

6. Conclusiones y Respuestas al Cuestionario

1. ¿Cuál es el estado general de calidad del código según SonarQube?

El estado general de calidad del código de **ComuniSafe** es **excelente**. Tras la aplicación de las mejoras, el sistema alcanzó la máxima calificación (**Rating A**) en Confiabilidad (Reliability), Seguridad (Security) y Mantenibilidad (Maintainability). Se erradicaron todos los Bugs y Code Smells reportados, llevando la deuda técnica estimada a cero minutos y manteniendo una tasa de duplicación saludable de 2.8%.

2. ¿Cuáles fueron los problemas más importantes encontrados y por qué?

Los problemas principales fueron: (1) **Incumplimiento de pautas de accesibilidad HTML5 (WCAG) en vistas Vue.js**, calificado como Bug por comprometer el uso del portal con lectores de pantalla; (2) **Alta complejidad cognitiva (21) en la función central de transición de estados de incidentes**, que concentraba lógica de autorización, derivación de tickets y notificaciones en un único bloque; y (3) **Duplicación repetitiva de literales clave** en modelos SQLAlchemy y routers de FastAPI, que amenazaba la consistencia del sistema ante cambios futuros.

3. ¿Qué cambios realizaron para corregirlos?

Se aplicaron tres técnicas estándar de refactorización: (a) Inclusión de etiquetas semánticas <caption class='sr-only'> en las tablas de VecinosView.vue y ReportsView.vue; (b) Extracción de constantes centralizadas (ERROR_REPORT_NOT_FOUND, CASCADE_ALL_DELETE, FK_USERS_ID, DEFAULT_MOCK_LOCATION); y (c) Modularización del endpoint update_report_status mediante la delegación en funciones auxiliares _validate_status_transition y _notify_status_update.

4. ¿Qué métricas mejoraron después de aplicar las correcciones?

- **Bugs:** Disminución de 2 a **0** (100% de bugs eliminados).
- **Reliability Rating:** Ascendió de **Rating B a Rating A**.
- **Code Smells:** Reducción de 6 a **0** (100% de olores resueltos).
- **Deuda Técnica:** Reducción de 53 min a **0 min**.
- **Duplicaciones:** Disminución de 2.9% a **2.8%**.
- **Cobertura Global:** Aumento de 49.9% a **50.1%**.

5. ¿Qué problemas siguen pendientes y qué acciones deberían realizarse en los próximos avances?

El área de mejora principal se encuentra en la **cobertura de pruebas automatizadas**: (a) Sincronizar y actualizar las pruebas de tests/test_backend.py implementando los endpoints reales de autenticación y registro para superar el 75-80% de cobertura en backend; (b) Incorporar una suite de pruebas unitarias para el frontend web con Vitest para emitir reportes lcov.info hacia SonarQube; y (c) Integrar la ejecución de SonarScanner en la canalización de integración continua (CI/CD) para evaluar cada Pull Request automáticamente antes de la integración.

---

7. Enlace al Repositorio e Instrucciones de Reproducción

El repositorio con el código fuente refactorizado y las configuraciones de escaneo se encuentra disponible en:
URL Git: https://github.com/MarceloRebolledo/ComuniSafePS.git
Las instrucciones paso a paso para levantar la instancia Docker y reproducir el escaneo de calidad están documentadas en el archivo README.md en la raíz del proyecto.